

Body Fit Analysis

1. Basic Theory

This project takes a full-body photo of a person, automatically detects the human body skeleton, measures the shoulder and hip width in centimeters, and predicts the most likely shirt and pant size. It also outputs a **Fit Confidence** percentage that tells us how reliable the measurements are. Everything runs locally in Python using just two main libraries: **MediaPipe** (for pose detection) and **OpenCV** (for image handling and drawing).

Pose Detection with MediaPipe

MediaPipe Pose is a pre-trained machine-learning model released by Google. Given an image of a person, it returns the (x, y, z) coordinates of **33 body landmarks** — points like nose, shoulders, elbows, wrists, hips, knees, and ankles. Each landmark also comes with a *visibility* score between 0 and 1 indicating how confident the model is that the point is actually visible in the image.

Key landmarks used in this project:

- **0** – Nose (top reference for height)
- **11, 12** – Left & right shoulders (shoulder width)
- **13, 14** – Left & right elbows
- **15, 16** – Left & right wrists
- **23, 24** – Left & right hips (hip width)
- **25, 26** – Left & right knees
- **27, 28** – Left & right ankles (bottom reference for height)

From Pixels to Centimeters

MediaPipe returns coordinates in *pixels*, but clothing sizes are measured in centimeters. To convert one to the other we use the person's **known real height** (passed with the `--height` flag). We measure the pixel distance from the nose down to the ankle midpoint, divide by the real height in cm, and get a pixels-per-cm ratio. This ratio is then used to convert shoulder and hip pixel widths into centimeters.

Width Measurements & Size Lookup

Shoulder width is the straight-line distance between landmarks **11** and **12**. Hip width is the distance between landmarks **23** and **24**. Once converted to cm, a simple threshold table maps the value to a size label (XS, S, M, L, XL for shirts; 28, 30, 32, 34... for pants). These tables sit at the top of the script and can be edited to match any brand's chart.

Fit Confidence

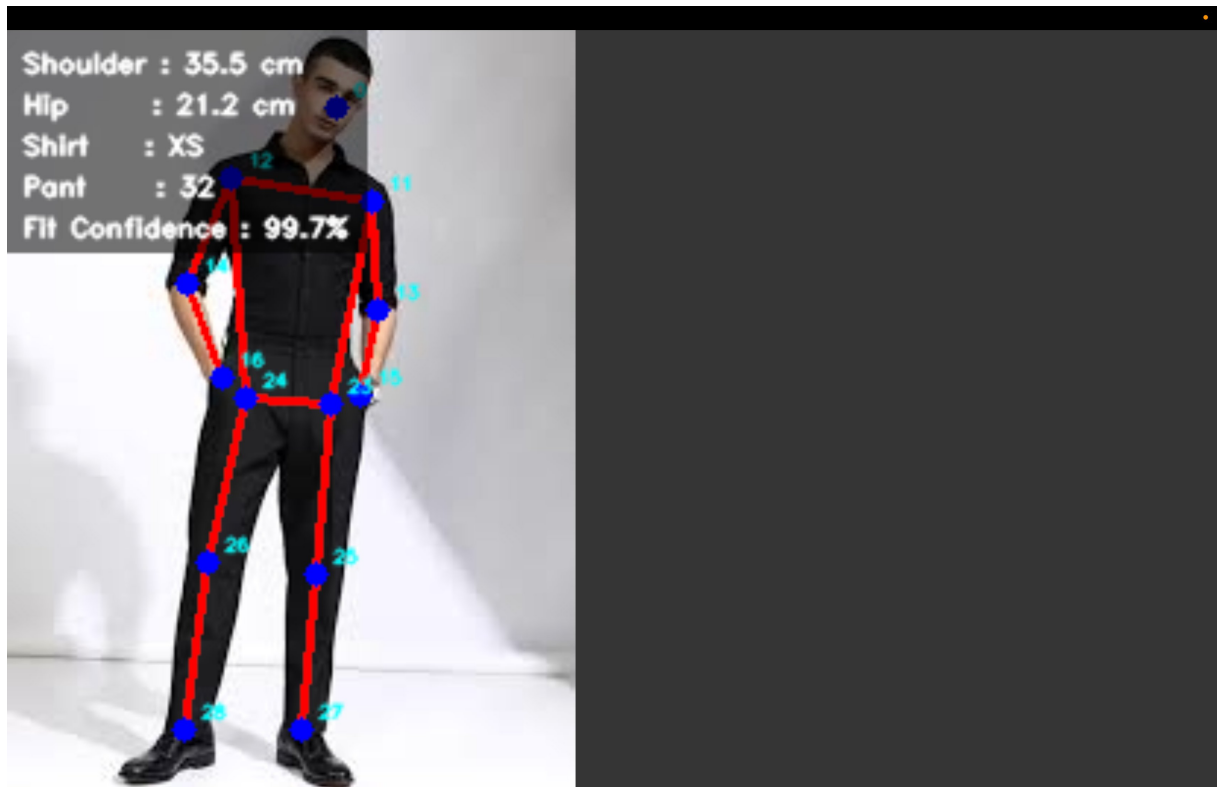
The Fit Confidence percentage is the **average visibility score** of the key landmarks: both shoulders, both hips, both ankles, and the nose. A score near 100% means every required landmark was clearly detected and the measurements are trustworthy. A low score warns the user to retake the photo.

Visualization

OpenCV draws the result back onto the image: red lines for skeleton connections, blue circles for landmarks, cyan numbers next to each dot, and a semi-transparent dark panel in the top-left with the five summary lines (Shoulder, Hip, Shirt, Pant, Fit Confidence). All sizes auto-scale based on image

dimensions so the output looks clean on any photo.

2. Output Screenshot



Sample output: Shoulder 35.5 cm, Hip 21.2 cm, Shirt XS, Pant 32, Fit Confidence 99.7%.

3. Complete Source Code

File: body_fit_analysis.py

```
"""
Body Fit Analysis (auto-scaling output)
Run: python body_fit_analysis.py --image photo.jpg --height 175
"""

import argparse
import math
import sys
import cv2
import mediapipe as mp
import numpy as np

mp_pose = mp.solutions.pose

NOSE = 0
L_SHOULDER, R_SHOULDER = 11, 12
L_ELBOW, R_ELBOW = 13, 14
L_WRIST, R_WRIST = 15, 16
L_HIP, R_HIP = 23, 24
L_KNEE, R_KNEE = 25, 26
L_ANKLE, R_ANKLE = 27, 28

SKELETON = [
    (L_SHOULDER, R_SHOULDER),
    (L_SHOULDER, L_ELBOW), (L_ELBOW, L_WRIST),
    (R_SHOULDER, R_ELBOW), (R_ELBOW, R_WRIST),
    (L_SHOULDER, L_HIP), (R_SHOULDER, R_HIP),
    (L_HIP, R_HIP),
    (L_HIP, L_KNEE), (L_KNEE, L_ANKLE),
    (R_HIP, R_KNEE), (R_KNEE, R_ANKLE),
]

DRAW_POINTS = [NOSE, L_SHOULDER, R_SHOULDER, L_ELBOW, R_ELBOW,
                L_WRIST, R_WRIST, L_HIP, R_HIP,
                L_KNEE, R_KNEE, L_ANKLE, R_ANKLE]

SHIRT_CHART = [(38, "XS"), (41, "S"), (44, "M"), (47, "L"),
               (50, "XL"), (53, "XXL"), (999, "XXXL")]

PANT_CHART = [(18, 28), (20, 30), (22, 32), (24, 34),
              (26, 36), (28, 38), (999, 40)]

def pick_size(value, chart):
    for thresh, size in chart:
        if value <= thresh:
            return size
    return chart[-1][1]

def to_px(lm, w, h):
    return int(lm.x * w), int(lm.y * h)

def euclid(p1, p2):
    return math.hypot(p1[0] - p2[0], p1[1] - p2[1])

def px_per_cm(landmarks, w, h, real_height_cm):
    nose = to_px(landmarks[NOSE], w, h)
    la = to_px(landmarks[L_ANKLE], w, h)
    ra = to_px(landmarks[R_ANKLE], w, h)
    ankle_mid = ((la[0] + ra[0]) / 2, (la[1] + ra[1]) / 2)
    person_px = euclid(nose, ankle_mid) * 1.12
    return person_px / real_height_cm if person_px > 0 else None

def resize_for_display(img, max_w=900, max_h=900):
    h, w = img.shape[:2]
    scale = min(max_w / w, max_h / h, 1.0)
    if scale < 1.0:
        img = cv2.resize(img, (int(w * scale), int(h * scale)),
                          interpolation=cv2.INTER_AREA)
    return img

def draw_text_panel(img, lines, scale):
```

```

font = cv2.FONT_HERSHEY_DUPLEX
font_scale = 0.6 * scale
thickness = max(1, int(1.5 * scale))
pad = int(10 * scale)
line_h = int(28 * scale)

widths = [cv2.getTextSize(t, font, font_scale, thickness)[0][0] for t in lines]
panel_w = max(widths) + pad * 2
panel_h = line_h * len(lines) + pad

overlay = img.copy()
cv2.rectangle(overlay, (0, 0), (panel_w, panel_h), (0, 0, 0), -1)
cv2.addWeighted(overlay, 0.55, img, 0.45, 0, img)

y = pad + int(line_h * 0.7)
for line in lines:
    cv2.putText(img, line, (pad, y), font, font_scale,
                (255, 255, 255), thickness, cv2.LINE_AA)
    y += line_h

def analyze(frame, pose, real_height_cm):
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    result = pose.process(rgb)

    if not result.pose_landmarks:
        cv2.putText(frame, "No person detected", (20, 60),
                    cv2.FONT_HERSHEY_SIMPLEX, 1.0, (0, 0, 255), 2)
        return frame

    lms = result.pose_landmarks.landmark
    h, w = frame.shape[:2]
    scale = max(0.5, min(h, w) / 800.0)
    dot_r = max(4, int(6 * scale))
    line_t = max(2, int(3 * scale))
    num_scale = 0.4 * scale
    num_thick = max(1, int(1 * scale))

    shoulder_px = euclid(to_px(lms[L_SHOULDER], w, h),
                        to_px(lms[R_SHOULDER], w, h))
    hip_px = euclid(to_px(lms[L_HIP], w, h),
                    to_px(lms[R_HIP], w, h))

    px_cm = px_per_cm(lms, w, h, real_height_cm)
    if not px_cm:
        return frame

    shoulder_cm = shoulder_px / px_cm
    hip_cm = hip_px / px_cm
    shirt = pick_size(shoulder_cm, SHIRT_CHART)
    pant = pick_size(hip_cm, PANT_CHART)

    key_ids = [L_SHOULDER, R_SHOULDER, L_HIP, R_HIP,
               L_ANKLE, R_ANKLE, NOSE]
    confidence = float(np.mean([lms[i].visibility for i in key_ids]))

    for a, b in SKELETON:
        cv2.line(frame, to_px(lms[a], w, h),
                 to_px(lms[b], w, h), (0, 0, 255), line_t)

    for idx in DRAW_POINTS:
        p = to_px(lms[idx], w, h)
        cv2.circle(frame, p, dot_r, (255, 0, 0), -1)
        cv2.putText(frame, str(idx), (p[0] + dot_r + 2, p[1] - dot_r),
                    cv2.FONT_HERSHEY_SIMPLEX, num_scale,
                    (255, 255, 0), num_thick, cv2.LINE_AA)

    lines = [
        f"Shoulder : {shoulder_cm:.1f} cm",
        f"Hip      : {hip_cm:.1f} cm",
        f"Shirt     : {shirt}",
        f"Pant      : {pant}",
        f"Fit Confidence : {confidence*100:.1f}%",
    ]
    draw_text_panel(frame, lines, scale)
    return frame

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--image", type=str, default=None)
    parser.add_argument("--height", type=float, default=170.0)

```

```

parser.add_argument("--save", type=str, default=None)
parser.add_argument("--cam", type=int, default=0)
args = parser.parse_args()

if args.image:
    img = cv2.imread(args.image)
    if img is None:
        print(f"Cannot open {args.image}"); sys.exit(1)
    with mp_pose.Pose(static_image_mode=True, model_complexity=2,
                      min_detection_confidence=0.5) as pose:
        out = analyze(img, pose, args.height)
    if args.save:
        cv2.imwrite(args.save, out)
        print(f"Saved -> {args.save}")
    out_display = resize_for_display(out, 900, 900)
    cv2.namedWindow("Body Fit Analysis", cv2.WINDOW_AUTOSIZE)
    cv2.imshow("Body Fit Analysis", out_display)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
else:
    cap = cv2.VideoCapture(args.cam)
    if not cap.isOpened():
        print("Cannot open webcam"); sys.exit(1)
    with mp_pose.Pose(static_image_mode=False, model_complexity=1,
                      min_detection_confidence=0.5,
                      min_tracking_confidence=0.5) as pose:
        while True:
            ok, frame = cap.read()
            if not ok: break
            frame = cv2.flip(frame, 1)
            out = analyze(frame, pose, args.height)
            cv2.imshow("Body Fit Analysis",
                      resize_for_display(out, 900, 900))
            if cv2.waitKey(1) & 0xFF == ord("q"):
                break
    cap.release()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()

```